

Lista 4 — Sistema de Autocomplete com Trie

1. Representação da Trie

O alfabeto escolhido tem tamanho 36, cobrindo letras minúsculas de 'a' a 'z' (26 caracteres) e dígitos de '0' a '9' (10 caracteres). Cada nó da Trie armazena um array de 36 ponteiros para filhos, um booleano indicando se aquele nó corresponde ao fim de um título, e um ponteiro para o objeto Game correspondente (caso seja nó terminal).

O mapeamento de caracteres para índices é feito pela função `charToIndex`: letras 'a'–'z' são mapeadas para os índices 0–25 e dígitos '0'–'9' para os índices 26–35. Caracteres fora desse intervalo retornam -1 e são tratados como inválidos.

Antes de qualquer operação de inserção ou busca, títulos e prefixos passam pela função `toSearchKey`, que converte todas as letras maiúsculas em minúsculas e remove espaços em branco. Assim, buscas como "Half Life", "halflife" e "HALF LIFE" são tratadas como equivalentes, tornando o sistema case-insensitive e indiferente a espaços.

2. Funcionamento dos Métodos

insert

O método converte o título do jogo para sua chave de busca e percorre a Trie caractere a caractere. Para cada caractere, verifica se o filho correspondente existe; caso não exista, cria um novo nó. Ao final da chave, marca o nó como fim de título e armazena o ponteiro para o objeto Game. A Trie não cria cópias dos jogos, os nós terminais apontam diretamente para os objetos do array fornecido.

contains

O método converte o título buscado para sua chave de busca e tenta seguir o caminho correspondente na Trie. Se em algum ponto o filho necessário não existir, retorna `false` imediatamente. Chegando ao fim da chave, retorna o valor do campo `isEndOfTitle`, garantindo que apenas títulos completos sejam reconhecidos, e não apenas prefixos.

autocomplete

O método converte o prefixo para sua chave de busca e navega até o nó correspondente na Trie. A partir desse nó, realiza uma busca em profundidade (DFS) pela função `collectGames`, que coleta todos os jogos encontrados na subárvore. Os resultados são então ordenados por popularidade decrescente — com desempate por ordem alfabética da chave de busca — usando `merge sort`. Por fim, o vetor é truncado para retornar no máximo `k` sugestões.

sortResults

O sistema de ordenação utiliza `merge sort`. O critério de ordenação é popularidade decrescente; em caso de empate, o desempate é feito pela ordem alfabética da chave de busca dos títulos. O `merge sort` foi escolhido por ter custo garantido $O(r \log r)$ em todos os casos, sem depender da ordem inicial dos elementos. Por fim, o vetor é truncado para retornar no máximo `k` sugestões.

3. Análise de Custo Computacional

insert: $O(L)$, onde L é o tamanho da chave de busca do título. O método percorre exatamente L nós, realizando operações de custo constante em cada um.

contains: $O(L)$. Percorre no máximo L nós antes de concluir, seja encontrando ou não o título.

autocomplete: $O(p + s + r \log r)$, onde p é o tamanho do prefixo convertido, s é o número de nós visitados na subárvore e r é o número de jogos encontrados. O custo $O(p)$ vem da navegação até o nó do prefixo; $O(s)$ vem da DFS em `collectGames`, que visita cada nó da subárvore uma vez; e $O(r \log r)$ vem da ordenação dos r resultados pelo merge sort.

4. Comparação e Uso de Memória

Uma solução ingênua armazenaria todos os jogos em um array e, para cada consulta de `autocomplete`, percorreria todos os N jogos verificando se o título começa com o prefixo dado. Isso resultaria em custo $O(N \cdot L)$ por consulta. Com a Trie, o custo de navegação até o prefixo é $O(p)$, independentemente do número total de jogos — uma vantagem significativa quando N é grande e os prefixos são curtos.

Em termos de memória, a Trie tem custo maior do que um simples array de jogos. Cada nó aloca um array de 36 ponteiros, independentemente de quantos filhos realmente existem. No pior caso, sem compartilhamento de prefixos, o número total de nós é proporcional à soma dos comprimentos das chaves de busca de todos os títulos.

O compartilhamento de prefixos é a principal vantagem estrutural da Trie: títulos com início comum — como "Hades", "Halo" e "Half Life" — compartilham os nós correspondentes ao prefixo "ha", reduzindo o número total de nós criados. Quanto maior a sobreposição entre os títulos, mais eficiente é o uso de memória.

O tamanho do alfabeto afeta diretamente o custo de memória por nó: com alfabeto de tamanho 36, cada nó aloca 36 ponteiros, independentemente de quantos são usados. Um alfabeto maior aumentaria esse custo fixo por nó, enquanto um alfabeto menor o reduziria. No caso deste projeto, o alfabeto de 36 caracteres é suficiente para cobrir títulos alfanuméricos sem desperdício excessivo.